

MVP SPECIFICATION & TEAM DELIVERY PLAN

ThirdWave

Minimum Viable Product — Functional Spec, UI Brief & Team Ownership

Single-Hazard, Single-District Pilot · 24 Screens · Version 1.0

Purpose	Defines exactly what the MVP includes, what it defers, and who on the team owns each part — the source of truth for AI-assisted UI generation and for sprint planning
Companion documents	ThirdWave Government–Citizen Product Specification (full V1) · ThirdWave UI/UX Design Specification (full V1, 41 screens)
Team	3-person founding team — Frontend & UI Engineer · Backend Engineer (Python/Django) · AI Engineer
MVP scope	One hazard family (flooding) · one pilot district · 24 of the 41 Version-1 screens
Date	August 2026

Table of Contents

1. Purpose & How to Use This Document
 2. MVP Rationale & Scope Boundary
 3. MVP Success Criteria
 4. MVP Functional Spec — Government
 5. MVP Functional Spec — Citizen
 6. MVP Functional Spec — Responder
 7. MVP Screen Inventory (24 Screens)
 8. Design System Quick Reference
 9. Data Model Overview
 10. MVP API Surface
 11. Risk Engine — MVP Approach
 12. Notifications & Alert Delivery — MVP Channels
 13. Team Roles & Responsibilities
 14. Responsibility Matrix (RACI)
 15. Delivery Plan — 10-Week Sprint Timeline
 16. Definition of Done — MVP Acceptance Criteria
- Appendix A — Deferred to Phase 2

1. Purpose & How to Use This Document

The full ThirdWave specification (65 functional sections, 41 screens) describes the complete national platform. It is not what a 3-person team should build first. This document defines the Minimum Viable Product: the smallest set of functionality that proves the entire operational loop — Detect → Understand → Decide → Warn → Act → Report → Respond → Learn — for one hazard, in one district, end to end, with real users on both sides of it.

This document has two audiences at once, deliberately: an AI design/coding agent generating the MVP interface, and the three-person team building it. Sections 4–12 are the functional and technical brief the AI should design from. Sections 13–16 are internal delivery planning — who owns what, in what order, and what 'done' means for this phase.

Instruction to the design agent

Generate only the 24 screens listed in Section 7, using the design system in Section 8 and the full visual language defined in the companion UI/UX Design Specification. Do not generate screens listed in Appendix A — they are explicitly out of scope for this phase.

2. MVP Rationale & Scope Boundary

Every cut below is deliberate and reversible — the underlying data model and architecture (companion spec, Technical Architecture) are built hazard-agnostic and role-extensible from day one, so nothing deferred here requires a redesign to add later. The MVP simply narrows what ships first.

2.1 What narrows, and why

Dimension	Full V1 spec	MVP	Why
Hazards	7 hazard types (rainfall, flash/urban/river flood, storms, dam, coastal)	Flooding only (heavy rainfall, flash, urban, river)	Proves the hazard-agnostic model with the highest-frequency, highest-impact hazard first.
Geography	National, multi-region, multi-district hierarchy	One pilot district	A real pilot with a real district office produces faster, more honest feedback than a simulated national rollout.
Government roles	8 distinct roles	3 roles: Operations Officer, Emergency Responder, Auditor (read-only, backend only)	National Administrator, Regional Officer, Analyst and Communications Officer permissions collapse into Operations Officer for one district; full role separation returns at national rollout.
Alert approval	Configurable multi-level workflow	Single-level: Draft → Publish	One district, one accountable officer — multi-level approval is a national-scale control, added at rollout.
Distribution channels	Push, SMS, USSD, web, email, partner feeds	Push notification + SMS	USSD requires telco short-code integration — a fast-follow, not a blocker to proving the loop.
Risk engine	Full ML risk model with regional performance tracking	Rule-based / threshold model over live rainfall + river data, with full provenance	An explainable, tunable rule-based model is faster to trust-test with a real district officer than an opaque ML model; Section 11 sets the roadmap to ML.
Analytics	Full Analytics module + historical event analysis	Live counts on the Command Centre only	Analytics needs weeks of real data to be meaningful — not useful in week 1 of a pilot.

2.2 What does not narrow

- Security: MFA, RBAC, TLS, encryption at rest, audit logging remain in scope from the first commit — these are not deferrable on a life-safety platform.
- Fail-safe behaviour: the platform must never silently show stale data, in the MVP or at any later stage.
- Citizen report verification: the human verification step remains mandatory — no shortcuts, even in a small pilot.
- Accessibility: WCAG 2.1 AA remains the bar for citizen-facing MVP screens.

3. MVP Success Criteria

The full platform targets thirteen real-time questions (companion spec §26). The MVP is successful if the pilot district can reliably answer the seven that depend only on the loop this phase builds:

1. What is happening? (current flood risk in the pilot district)
2. Where is it happening? (which specific areas, on the map)
3. How severe is the risk? (Low / Watch / Warning / Critical, with confidence)
4. Did citizens receive the warning? (delivery status per alert)
5. Who is responding? (assigned responder, live status)
6. What is the current situation? (incident status in real time)
7. Has the incident been resolved?

Two additional, purely operational MVP success measures:

- Time from a government-published alert to a citizen receiving it, measured end to end, target under 2 minutes for push and under 5 minutes for SMS.
- At least one full loop cycle completed live during the pilot: risk detected → alert published → citizen notified → citizen report submitted → verified → incident created → responder dispatched → resolved.

4. MVP Functional Spec — Government

4.1 Roles in scope

Role	MVP permissions
Operations Officer	Full district operational access: view risk map, create and publish alerts, verify citizen reports, create and assign incidents, manage shelters (read/write), view live Command Centre counts.
Emergency Responder	View and accept assigned incidents, update status, upload evidence, mark resolved — via the Responder app only.
Auditor (backend only)	Audit log records are captured from day one; a dedicated Audit Log screen is deferred (Appendix A), but the data is queryable directly for compliance if needed during the pilot.

4.2 Core capabilities

- View a live district risk map for flood-related layers: rainfall, flood risk zones, river levels, active warnings.
- Open a Risk Detail Drawer for any affected area: risk summary, contributing factors, potential impact, recommended actions, data provenance.
- Create and publish a flood alert (Advisory / Watch / Warning / Emergency) to the pilot district, with a live preview of what the citizen will see.
- Receive, review and verify citizen-submitted flood reports; a verified report becomes an official Incident with a unique ID.
- Assign a verified incident to an available Emergency Responder and track its status through to Resolved.
- View and edit a basic shelter register (name, location, capacity, occupancy, status) for the pilot district.
- See six live Command Centre counts: Active Alerts, District Risk Level, Active Incidents, Citizens Reached, Responders Active, Shelters Available.

5. MVP Functional Spec — Citizen

- See current risk for their location within three seconds of opening the app (Low / Watch / Warning / Critical), plus one recommended action.
- Receive push and/or SMS alerts for their saved/current location.
- View a simplified risk map: current location, flood zones, official warnings, shelters.
- Report a flood incident with photo, description and auto-captured GPS location.
- See a confirmation that a report was submitted and is 'Under Review' — never implying it is already official.
- Find the nearest emergency shelter with basic directions.
- Set basic preferences: language (English at launch), notification on/off, and reach the plain-language privacy notice at any time.

Deferred for MVP: saved multiple locations beyond current + one home location, family safety / trusted contacts, community reports feed, and a dedicated notifications inbox (alerts surface directly on Home for MVP). See Appendix A.

6. MVP Functional Spec — Responder

- Log in and see a list of assigned incidents, sorted by severity.
- Open an incident to see hazard, severity, location, description and citizen-submitted evidence.
- Navigate to the incident location via a map with a route line.
- Update status through four large, thumb-reachable actions: Accept → En Route → On Scene → Resolved.
- Capture and upload evidence (photo) at the scene, with basic offline queuing if connectivity drops — synced automatically once reconnected.

Deferred for MVP: full offline map tiles, vehicle/equipment assignment visibility, and in-app messaging with the Command Centre (a phone-call fallback is used for the pilot).

7. MVP Screen Inventory (24 Screens)

This is the complete, bounded MVP build. Generate exactly these 24 screens, using the layout frames, components and screen-spec patterns defined in the companion UI/UX Design Specification. Any screen not listed here is out of scope for this phase (Appendix A).

7.1 Government (9 screens)

#	Screen	MVP notes
1	Login	Email/password + MFA. Single district context — no organisation switcher needed.
2	Command Centre Dashboard	6 KPI cards (Section 4.2) + mini risk map + recent alerts/incidents feed.
3	Risk Map	Flood-relevant layers only: rainfall, flood risk, river levels, warnings, shelters.
4	Risk Detail Drawer	Opens over the map. Full risk summary, contributing factors, impact, actions, provenance.
5	Alerts Dashboard	List + status (Draft/Published/Expired). 'New Alert' primary action.
6	Create Alert	Single-level: Draft → Publish. Live citizen-view preview panel.
7	Alert Detail	Message, delivery status/rate, Cancel action. No multi-level approval UI in MVP.
8	Incidents Dashboard	Includes the citizen-report verification queue (Submitted / Under Review states).
9	Incident Detail	Verification action, evidence, assign responder, Incident Status tracker, resolve.

7.2 Citizen (9 screens)

#	Screen	MVP notes
10	Onboarding	3-step intro, sets expectations plainly (what ThirdWave does / does not promise).
11	Location Permission	Allow-location or manual-entry fallback.
12	Home	Risk hero + recommended action + shortcuts. Also carries current alerts (no separate inbox in MVP).
13	Alert Detail	Full-screen takeover for Emergency type; card view for lower severities.
14	Risk Map	Simplified layer set: location, flood zones, warnings, shelters.
15	Report Incident	Hazard-type icon grid limited to flood-relevant types; photo + GPS + description.
16	Report Confirmation	States 'Submitted — Under Review' with a Mono reference ID.
17	Emergency Shelter	List sorted by distance + detail view with directions.
18	Settings	Language, notifications toggle, privacy notice link. Combines Profile for MVP.

7.3 Responder (6 screens)

#	Screen	MVP notes
19	Login	Same pattern as Government Login, mobile single-column.
20	Responder Dashboard	Assigned incidents list + availability toggle.
21	Incident Detail	Hazard, evidence, citizen contact where authorised, pinned status actions.
22	Navigation / Map	Route line to incident location, ETA.
23	Status Update	Accept / En Route / On Scene / Resolved — large buttons, one active at a time.
24	Evidence Upload	Camera-first capture with offline queue indicator.

8. Design System Quick Reference

Full detail — every component, state, and layout frame — lives in the companion UI/UX Design Specification. This is the minimum an AI agent needs if generating from this document alone.

8.1 Colour

Token	Hex	Usage
ink-navy (brand)	#0B2545	Government chrome, headings, primary buttons
steel-blue	#1E4E6B	Secondary structural colour
signal-teal (accent)	#1B7A9E	Links, focus rings, the Detection Ring motif
paper / surface	#F7F9FA / #FFFFFF	App background / cards
risk green / amber / orange / red	#1E8A4C / #B8860B / #C1621B / #B0241D	Low / Watch / Warning / Critical — colour always paired with a text label

8.2 Typography

IBM Plex Sans for interface and reading text (Bold for headings, Regular/Medium for body). IBM Plex Mono for IDs, timestamps, coordinates and counters. No other families.

8.3 Layout frames

Government: 12-column desktop frame, fixed header + sidebar (see companion spec §10.1). Citizen: mobile-first single column with bottom tab bar (§10.2), reduced to 4 tabs for MVP — Home, Map, Report, Shelters. Responder: mobile-first, one thumb-reachable primary action per screen (§10.3).

8.4 Non-negotiable rules carried into the MVP

- Never show a risk or status colour without a text label.
- Never silently show stale data — always disclose source and last-update time where risk-relevant.
- Always visually distinguish Official / Predicted / Verified / Unverified content (companion spec §8.3).

9. Data Model Overview

Indicative core entities for the Django backend. Field lists are illustrative, not exhaustive — the backend engineer owns the final schema, but every entity below must exist in some form for the MVP loop to function end to end.

Entity	Core fields	Notes
User	id, name, phone/email, password_hash, role, district_id, mfa_enabled	Role $\in$ {ops_officer, responder, citizen}. Citizens may be anonymous/phone-verified rather than full accounts for MVP speed.
District	id, name, boundary (GeoJSON), centre_point	Single row for the pilot, modelled as many from day one.
RiskZone	id, district_id, hazard_type, geometry, level, confidence, contributing_factors[], valid_from, valid_to, model_version	Written by the Risk Engine service (Section 11); read by both government and citizen map layers.
Alert	id, hazard_type, severity_type (advisory/watch/warning/emergency), area (geometry), message, recommended_action, status (draft/published/expired/cancelled), created_by, published_at, expires_at	Single-level workflow for MVP: draft $\leftrightarrow$ published.
AlertDelivery	id, alert_id, channel (push/sms), recipient_count, delivered_count, status	Populated by the notification dispatch service; powers the delivery-rate MVP success metric.
CitizenReport	id, reporter_ref, hazard_type, description, photo_url, gps_point, status (submitted/under_review/verified/rejected), submitted_at	reporter_ref may be a phone number rather than a full user id for MVP.
Incident	id (INC-GH-2026-#####), report_id (nullable), hazard_type, location, severity, status (reported $\rightarrow$ closed), assigned_responder_id, response_notes, created_at, resolved_at	Created automatically on report verification, or manually by an Operations Officer.
Shelter	id, district_id, name, location, capacity, occupancy, contact, status	Simple CRUD for MVP; no accessibility/supplies detail fields yet.
AuditLogEntry	id, actor_id, action, target_type, target_id, timestamp, metadata	Written automatically by middleware on every sensitive action, from commit one, even without a dedicated viewer screen in MVP.

Recommended stack note for the backend engineer: PostgreSQL with PostGIS for all geometry fields (RiskZone, Alert.area, CitizenReport.gps_point, Shelter.location), Django REST Framework for the API layer, Django's built-in auth extended with the role field and MFA via a TOTP package.

10. MVP API Surface

Grouped by companion-spec API category (§36). Full request/response contracts are defined jointly by the Backend Engineer and Frontend Engineer during Sprint 1 (Section 15); this is the agreed surface, not the final contract.

Category	MVP endpoints (indicative)
Auth	POST /auth/login, POST /auth/mfa/verify, POST /auth/refresh
Risk	GET /risk/zones?district=, GET /risk/zones/{id} (detail + provenance)
Alerts	GET /alerts, POST /alerts, GET /alerts/{id}, POST /alerts/{id}/publish, POST /alerts/{id}/cancel
Reports	POST /reports (citizen submit), GET /reports?status=, POST /reports/{id}/verify, POST /reports/{id}/reject
Incidents	GET /incidents, GET /incidents/{id}, POST /incidents/{id}/assign, POST /incidents/{id}/status, POST /incidents/{id}/evidence
Shelters	GET /shelters?district=, POST /shelters, PATCH /shelters/{id}
Notifications	GET /notifications (citizen), POST /devices/register (push token)
System	GET /system/health

11. Risk Engine — MVP Approach

The MVP risk engine is deliberately a transparent, rule-based model rather than a trained ML model — faster to build, faster to validate with a real district officer, and fully explainable, which matters more than raw accuracy at this stage.

11.1 MVP model logic (indicative)

- Ingest two live feeds for the pilot district: a rainfall feed (forecast + observed) and a river-level feed, polled on a fixed interval.
- Apply threshold rules per zone: e.g. rainfall > X mm in 6h AND river level > Y → Warning; add a Critical tier when both exceed a second, higher threshold or a dam-release flag is present.
- Every output RiskZone row carries its contributing factors and source feeds verbatim (companion spec §10, §34) — this is not optional, it is how the interface earns trust.
- Recompute on a short interval (e.g. every 15 minutes) and always timestamp the output so the fail-safe UI rule (§8.4) can be enforced by the frontend.

11.2 Roadmap beyond MVP

Once the rule-based model has run through a full rainy season with logged outcomes, the AI Engineer's next phase is a supervised model (e.g. gradient-boosted or a simple hydrology-informed regression) trained against verified incident outcomes, with the existing rule-based model retained as an explainable fallback and as the baseline the ML model must beat before replacing it — this satisfies the companion spec's Prediction Performance requirement (§33) from day one.

12. Notifications & Alert Delivery — MVP Channels

- Push notifications via a standard mobile push provider (e.g. Firebase Cloud Messaging) for the citizen app.
- SMS via a single local aggregator/gateway integration for citizens without the app installed or without data connectivity at the moment of alert.
- USSD is explicitly deferred (Section 2.1) — it requires a telco short-code agreement that is a procurement dependency, not an engineering one, and should be pursued in parallel during the pilot so it is ready for national rollout.
- Every AlertDelivery record tracks recipient_count and delivered_count per channel, so the delivery-rate MVP success metric (Section 3) is measurable from the first alert sent.

13. Team Roles & Responsibilities

Three roles, three people, deliberately no overlap in primary ownership — each person is the single accountable owner of their layer, while all three share responsibility for the integration points between them (Section 14).

13.1 Frontend & UI Engineer

Mission

Turn this specification and the companion UI/UX Design Specification into the 24 production screens across Government, Citizen and Responder — pixel-accurate to the design system, fully responsive, and accessible.

- Own the shared component library (Alert Banner, Risk Badge, Risk Card, Map, KPI Card, Data Table, Incident Status, etc. — companion spec §9) as reusable, framework-level components.
- Build all 9 Government screens (desktop web), 9 Citizen screens (mobile-first, responsive web fallback) and 6 Responder screens (mobile).
- Implement the Detection Ring signature element, risk colour system, and all fail-safe/empty/loading/error states specified for each screen.
- Integrate with the Backend Engineer's API surface (Section 10); own client-side state management, offline queuing UI for the Responder app, and push-notification client wiring.
- Own accessibility: WCAG 2.1 AA pass on every citizen-facing screen before MVP sign-off.
- Deliverables: a working design-system component library; 24 functioning screens wired to real APIs; a responsive/accessibility QA pass.

13.2 Backend Engineer (Python / Django)

Mission

Build the data model, business logic and API surface that make the loop real — authentication, alerts, incidents, reports, shelters, notifications and audit logging.

- Own the Django project: data models (Section 9), migrations, and Django REST Framework API layer (Section 10).
- Implement authentication, RBAC for the 3 MVP roles, and MFA.
- Implement the Alert lifecycle (draft/publish/cancel), the Citizen Report verification state machine, and the Incident lifecycle including responder assignment.
- Integrate the push notification provider and SMS gateway, and implement the AlertDelivery tracking used for the MVP success metric.
- Implement audit-log middleware so every sensitive action is recorded from the first commit, per the companion spec's non-negotiable rule (§38).
- Own deployment basics: environment configuration, database (PostgreSQL/PostGIS), backups, and the /system/health endpoint the frontend's fail-safe UI depends on.
- Deliverables: a documented, versioned REST API; a seeded pilot-district database; MFA/RBAC in place; automated backups running before pilot go-live.

13.3 AI Engineer

Mission

Turn raw rainfall and river data into the explainable RiskZone output the rest of the platform depends on, and keep it honest about its own confidence.

- Own the data ingestion pipeline for the two MVP feeds (rainfall, river level) for the pilot district, including handling gaps/outages per the fail-safe rule.
- Build and tune the MVP rule-based flood risk model (Section 11.1), and the recompute/scheduling job that keeps RiskZone rows current.
- Ensure every model output carries full provenance (sources, timestamp, model/version) as a first-class field, not an afterthought — this directly feeds the Risk Detail Drawer and citizen 'why' explanations.
- Log verified incident outcomes against prior model predictions from day one, building the labelled dataset the post-MVP ML model (Section 11.2) will need.
- Own the /risk/zones API contract jointly with the Backend Engineer, and the model-confidence values the frontend's 'reduced confidence' fail-safe state depends on.
- Deliverables: a running ingestion pipeline with monitoring/alerting on feed failure; a documented rule-based model with a written explanation of every threshold chosen; a logged prediction-vs-outcome dataset from week one of the pilot.

14. Responsibility Matrix (RACI)

R = Responsible (does the work) A = Accountable (owns the outcome) C = Consulted I = Informed

Module	Frontend & UI	Backend (Django)	AI Engineer
Design system & component library	R/A	I	I
Government screens (9)	R/A	C	I
Citizen screens (9)	R/A	C	I
Responder screens (6)	R/A	C	I
Auth, RBAC & MFA	C	R/A	I
Alert lifecycle & delivery tracking	C	R/A	I
Citizen report & incident workflow	C	R/A	I
Shelter register	C	R/A	I
Audit logging	I	R/A	I
Push/SMS notification integration	C	R/A	I
Data ingestion (rainfall/river feeds)	I	C	R/A
Risk engine (rule-based model)	C	C	R/A
Risk/provenance API contract	C	R/A	R/A
System health endpoint & fail-safe UI	R	R/A	C
Accessibility (WCAG 2.1 AA)	R/A	I	I
Pilot data & deployment infrastructure	I	R/A	C

15. Delivery Plan — 10-Week Sprint Timeline

Sprint	Frontend & UI	Backend (Django)	AI Engineer
1-2 Foundations	Design tokens + component library scaffolding; static Login, Command Centre, Home screens	Django project setup, data models, auth/RBAC/MFA, API scaffolding	Ingestion pipeline for rainfall + river feeds; provenance schema agreed with Backend
3-4 Risk & Map	Risk Map, Risk Detail Drawer, Citizen Risk Map — wired to live /risk/zones	Alert model + endpoints; Shelter CRUD	Rule-based model v1 live; first RiskZone rows in the district
5-6 Alerts	Alerts Dashboard, Create Alert, Alert Detail; Citizen Alert Detail + push/SMS receipt	Alert publish/cancel logic; push + SMS gateway integration; AlertDelivery tracking	Model tuning against first live rainfall events; confidence scoring finalised
7-8 Reports & Incidents	Report Incident, Report Confirmation, Incidents Dashboard, Incident Detail; Responder Dashboard + Incident Detail + Status Update + Evidence Upload	Citizen report verification workflow; incident lifecycle + assignment; offline evidence sync endpoint	Begin logging prediction-vs-outcome pairs; feed-failure monitoring in place
9 Hardening	Accessibility pass (WCAG 2.1 AA); empty/error/loading states across all 24 screens; System Health banner	Audit logging complete; backups + basic disaster-recovery test; security pass (TLS, secrets, rate limiting)	Provenance/ confidence values reviewed end-to-end with the officer who will use them
10 Pilot go-live	Final UAT with district officer, responder and citizen testers; bug-fix buffer	Production deploy to pilot environment; monitoring/alerting live	Model live-monitoring dashboard for the AI Engineer's own use during the pilot

Each sprint ends with a short three-way demo — every screen shown against a live API response, not a mock — so integration gaps surface weekly rather than in week 10.

16. Definition of Done — MVP Acceptance Criteria

8. All 24 screens in Section 7 are live, wired to real APIs (no mocked data remaining), and pass the companion Design Specification's Definition of Done (§21).
9. A single alert can be created, published, and its delivery tracked end to end across push and SMS.
10. A citizen report can be submitted, verified by an Operations Officer, converted to an Incident, assigned to a Responder, and marked Resolved — with every state transition visible in the correct screen.
11. The Risk Map and Risk Detail Drawer reflect live model output, including provenance and a visible 'last updated' time.
12. MFA, RBAC and audit logging are active in the pilot environment, not just in a development environment.
13. The fail-safe rule is demonstrably enforced: disconnecting a data feed produces the 'source unavailable' state, not stale-looking live data.
14. WCAG 2.1 AA verified on all 9 citizen screens.
15. The two MVP success metrics in Section 3 are measurable from the platform's own data, not from manual observation.

Appendix A — Deferred to Phase 2

Explicitly out of scope for the MVP. Nothing here requires re-architecture to add later — it is sequencing, not redesign.

- Government: Response Management (dedicated screen), Critical Infrastructure registry, Analytics module, System Health (dedicated screen), User Management, Audit Logs (viewer), Integrations, multi-level alert approval, Regional/National role hierarchy.
- Citizen: Current Risk (separate screen), Community Reports, Saved Locations (multiple), Notifications (dedicated inbox), Family Safety, Profile (separate from Settings).
- Responder: in-app messaging, vehicle/equipment assignment visibility, full offline map tiles.
- Platform-wide: USSD channel, additional hazard types beyond flooding, additional languages beyond English, AI Assistant for Government, ML-based risk model (rule-based model continues through the pilot), private-sector product extension.

END OF MVP SPECIFICATION — ThirdWave — Version 1.0